

UNIVERSIDAD POLITECNICA DE SAN LUIS POTOSI



Ingeniería en tecnologías de la información

Actividad 12 ROAD TO HALL OF FAME

*LAB: FILE PATH TRAVERSAL VALIDATION OF FILE
EXTENSION WITH NULL BYTE BYPASS*

Alumno: Isaac Elihu Rios de la Tejera

Docente: Servando López Contreras

Asignatura: CNO IV Seguridad Informática

Fecha de entrega: 28/09/2026

1. Nombre del laboratorio

File path traversal, validation of file extension with null byte bypass

2. Nivel

Apprentice (Aprendiz)

3. Tipo de explotación

Path Traversal / Directory Traversal mediante Null Byte Injection.

La explotación combina secuencias de recorrido de directorios como:

../

con la inyección de un **byte nulo**, representado mediante URL Encoding como:

%00

El objetivo de esta técnica es evadir la validación que obliga a que el nombre solicitado termine con una extensión de imagen válida, como .jpg.

4. Objetivo del laboratorio

El objetivo del laboratorio consiste en explotar una vulnerabilidad de **Path Traversal** presente en el parámetro:

filename

utilizado por la aplicación para recuperar imágenes almacenadas en el servidor.

La aplicación aplica una restricción que exige que el valor recibido termine en una extensión determinada, como:

.jpg

Para superar esta validación se utiliza un **Null Byte Bypass**, permitiendo solicitar el archivo:

/etc/passwd

sin perder la extensión requerida por la aplicación durante la validación.

El objetivo final es demostrar que una validación basada únicamente en la extensión del nombre del archivo puede resultar insuficiente cuando existen diferencias entre cómo distintas capas de la aplicación procesan una misma cadena.

5. Explicación técnica de la vulnerabilidad

La aplicación utiliza el parámetro filename para indicar qué archivo debe recuperar del servidor.

Una petición legítima puede tener una estructura similar a:

GET /image?filename=51.jpg HTTP/2

Como mecanismo de seguridad, la aplicación comprueba que el valor proporcionado termine con una extensión de imagen permitida, por ejemplo: .jpg

Esto impide que una petición directa como:

../../../../etc/passwd

sea aceptada, ya que no termina con la extensión requerida.

Sin embargo, la aplicación también presenta dos problemas de seguridad.

El primero es que permite introducir secuencias de recorrido de directorios: ../

lo que hace posible desplazarse fuera del directorio destinado a almacenar imágenes.

El segundo problema es que el procesamiento del nombre del archivo permite utilizar un carácter nulo representado como:

%00

Después de la decodificación URL, este valor representa el carácter **NUL**.

En ciertos entornos vulnerables, las distintas capas de la aplicación pueden interpretar la misma cadena de forma diferente. La lógica de validación puede comprobar una cadena como:

../../../../etc/passwd%00.jpg

y detectar que termina en:

.jpg

por lo que considera que cumple con la condición establecida.

Sin embargo, una capa inferior utilizada posteriormente para acceder al sistema de archivos puede interpretar el carácter NUL como el final efectivo de la cadena.

Conceptualmente:

../../../../etc/passwd%00.jpg

|

NULL

puede terminar siendo procesado como:

../../../../etc/passwd

De esta manera, la extensión .jpg permite superar la validación inicial, mientras que el procesamiento posterior utiliza únicamente la parte situada antes del byte nulo.

El resultado es el acceso a un archivo que se encuentra fuera del directorio autorizado.

6. Procedimiento paso a paso

1. Acceder al laboratorio correspondiente desde **PortSwigger Web Security Academy**.
2. Iniciar **Burp Suite** y utilizar el navegador configurado para enviar el tráfico mediante el proxy.
3. Dentro de la tienda, seleccionar cualquier producto para provocar una solicitud de carga de imagen.
4. En **Burp Suite** → **Proxy** → **HTTP history**, localizar una petición similar a:

GET /image?filename=51.jpg HTTP/2

5. Enviar la solicitud a **Burp Repeater** utilizando:

Send to Repeater

6. En Repeater, enviar inicialmente la petición sin modificar para comprobar el comportamiento normal de la aplicación.
7. Modificar el parámetro filename utilizando una secuencia de recorrido de directorios que permita alcanzar el archivo objetivo.

En esta ejecución se utilizó:

../../../../etc/passwd%00.jpg

8. La solicitud modificada queda conceptualmente de la siguiente forma:

GET /image?filename=../../../../etc/passwd%00.jpg HTTP/2

9. Presionar el botón **Send**.
10. Analizar la respuesta obtenida en Burp Repeater.

Si la explotación se realiza correctamente, en el cuerpo de la respuesta aparecerá información perteneciente a:

/etc/passwd

por ejemplo:

root:x:0:0:root:/root:/bin/bash

11. Regresar al navegador del laboratorio.
12. Verificar que PortSwigger muestre el mensaje:

Congratulations, you solved the lab!

y que el estado del laboratorio aparezca como:

Solved

7. Capturas de pantalla obligatorias

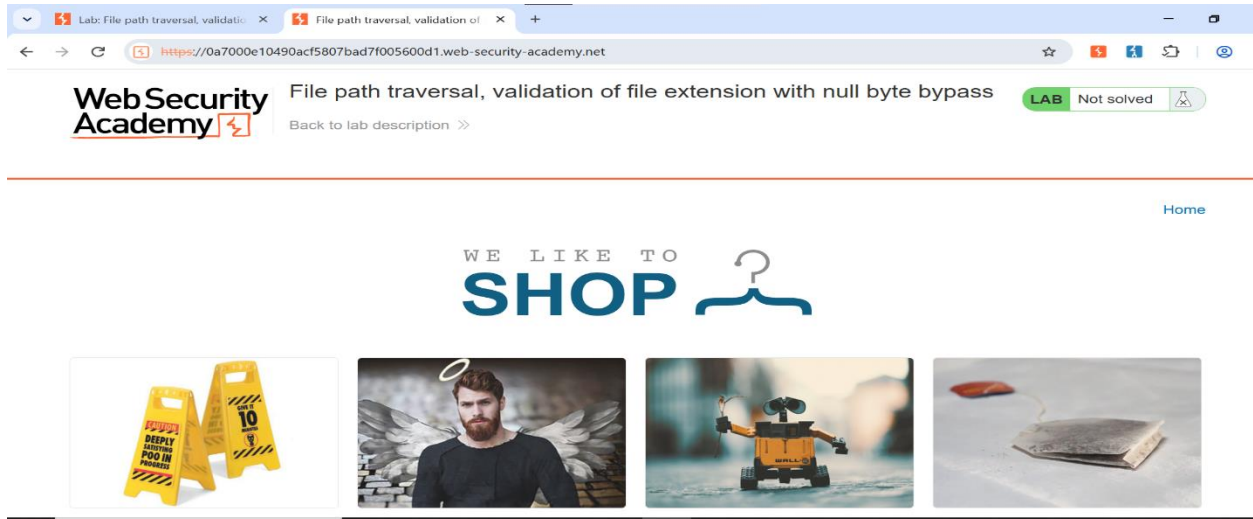


Ilustración 1 Estado inicial del laboratorio

Captura del navegador mostrando el laboratorio antes de realizar cualquier modificación o explotación.

Debe observarse el estado:

Not solved

Esta evidencia permite demostrar que el laboratorio todavía no había sido resuelto al inicio de la práctica.

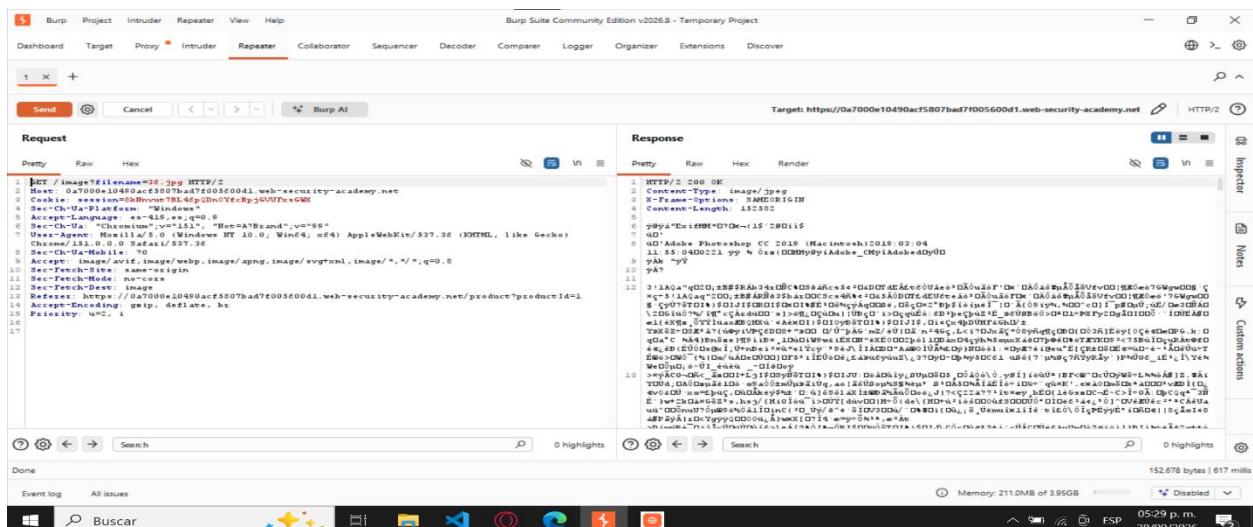


Ilustración 2 Request original interceptado

Captura de Burp Suite mostrando la petición legítima generada por la aplicación al solicitar una imagen.

Por ejemplo:

GET /image?filename=51.jpg HTTP/2

Esta captura permite identificar el parámetro vulnerable:

filename

y observar el comportamiento normal de la aplicación antes de modificar la solicitud.

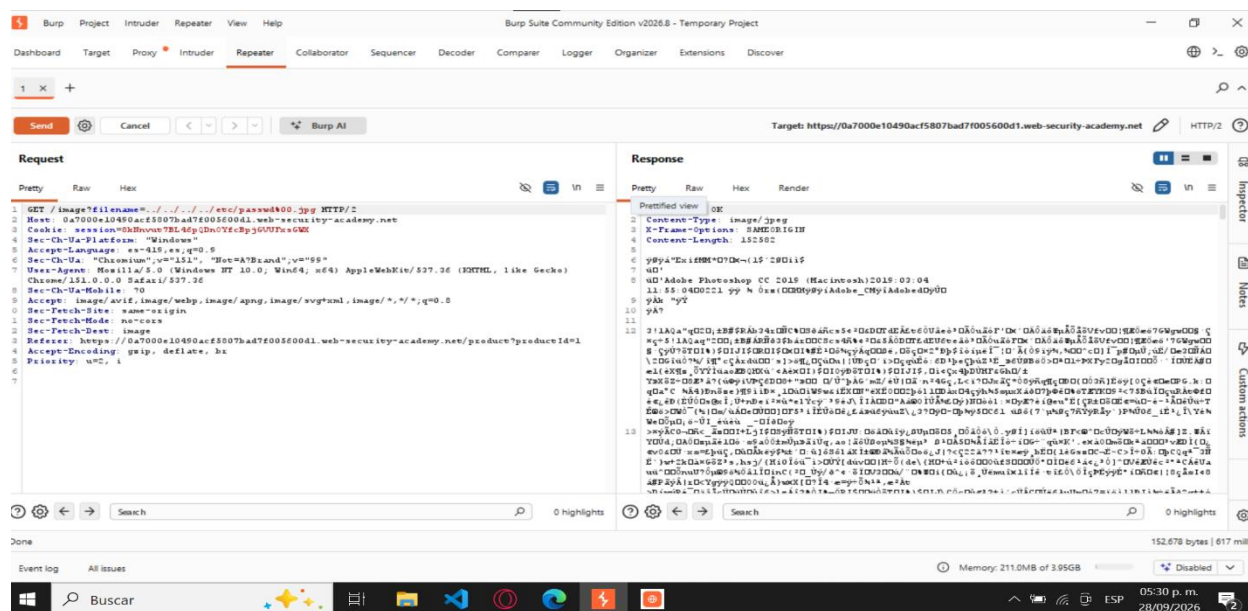


Ilustración 3 Payload utilizado

Captura de **Burp Repeater** mostrando la petición modificada con el payload empleado para realizar la explotación:

GET /image?filename=.././.././etc/passwd%00.jpg HTTP/2

En esta evidencia debe observarse claramente la combinación de:

.././.././

para realizar el recorrido de directorios,

etc/passwd

como archivo objetivo,

%00

como byte nulo utilizado para evadir la validación, y:

.jpg

como extensión utilizada para satisfacer la comprobación de la aplicación.

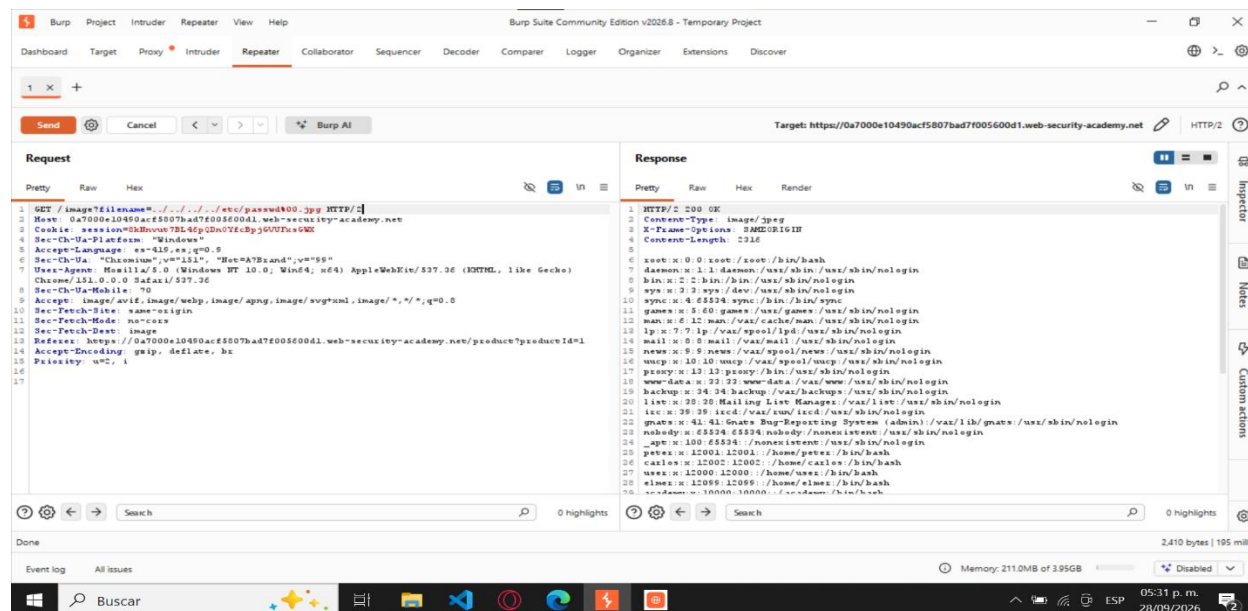


Ilustración 4 Respuesta vulnerable

Captura de la sección **Response** de Burp Repeater después de enviar la petición modificada.

En el cuerpo de la respuesta debe observarse contenido correspondiente al archivo:

/etc/passwd

por ejemplo:

root:x:0:0:root:/root:/bin/bash

Esta evidencia confirma que el servidor procesó la ruta manipulada y permitió acceder a un archivo ubicado fuera del directorio destinado a las imágenes.

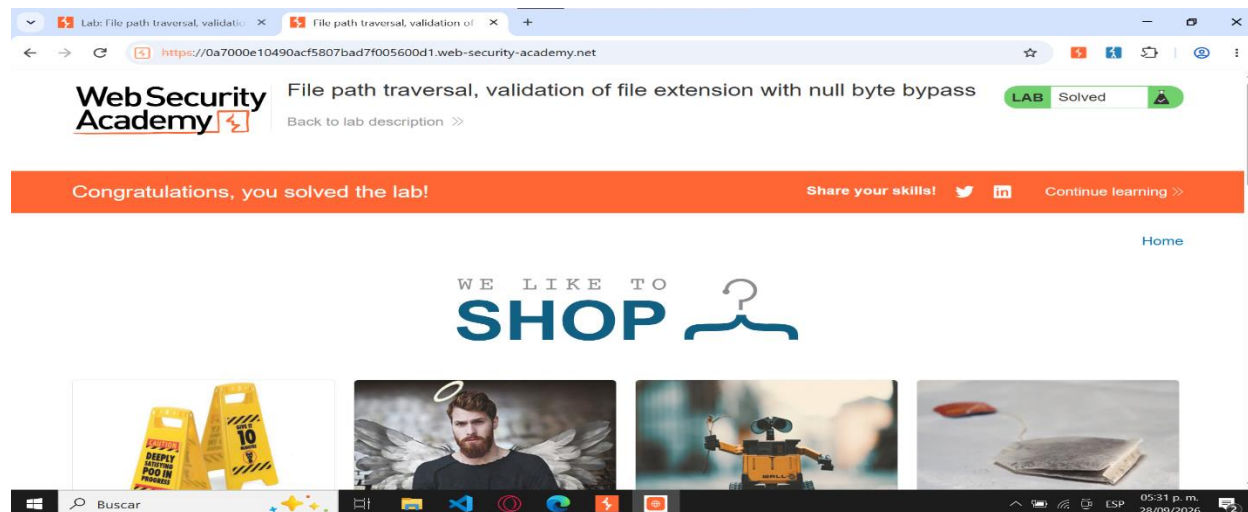


Ilustración 5 Confirmación de "Lab Solved"

Captura final del navegador mostrando que PortSwigger reconoció correctamente la explotación.

Debe observarse el mensaje:

Congratulations, you solved the lab!

junto con el estado:

Solved

Esta captura confirma que el laboratorio fue completado satisfactoriamente.

8. Explicación del payload

El payload utilizado fue:

../../../../etc/passwd%00.jpg

Puede dividirse en cuatro componentes principales.

Secuencia de recorrido de directorios

../../../../

Cada secuencia:

../

indica al sistema de archivos que debe desplazarse hacia el directorio padre.

Al utilizar varias secuencias consecutivas se intenta salir del directorio en el que la aplicación almacena las imágenes hasta alcanzar una ubicación desde la cual sea posible acceder al archivo objetivo.

Archivo objetivo

etc/passwd

Corresponde al archivo solicitado en el sistema Linux.

La ruta final que se pretende alcanzar es:

/etc/passwd

Este archivo contiene información de cuentas del sistema, aunque las contraseñas modernas normalmente no se almacenan directamente en él.

Byte nulo

%00

%00 es la representación mediante URL Encoding del carácter NUL.

Este elemento constituye la parte principal del bypass.

La aplicación recibe conceptualmente:

../../../../etc/passwd%00.jpg

y su validación puede comprobar que la cadena termina en:

.jpg

Sin embargo, una capa vulnerable utilizada posteriormente para manejar la ruta puede interpretar el NUL como final de cadena.

De manera simplificada:

../../../../etc/passwd%00.jpg

		└─ Parte utilizada para superar
		la validación de extensión
		└─ Ruta realmente procesada por la capa vulnerable

Extensión .jpg

.jpg

La extensión se añade para cumplir con la validación implementada por la aplicación.

Por lo tanto, existe una diferencia entre lo que observa la validación:

../../../../etc/passwd%00.jpg

y lo que puede terminar utilizando el componente vulnerable encargado de acceder al archivo:

../../../../etc/passwd

9. Mitigación recomendada

Para prevenir este tipo de vulnerabilidad se recomienda implementar múltiples controles de seguridad.

- **Evitar utilizar directamente rutas proporcionadas por el usuario:** en lugar de recibir nombres o rutas completas, la aplicación debería utilizar identificadores internos, por ejemplo id=51, y asociarlos en el servidor con archivos previamente autorizados.
- **Aplicar listas blancas:** únicamente deben aceptarse nombres de archivos previamente permitidos. Validar solamente la extensión no garantiza que la ruta sea segura.
- **Normalizar y canonicalizar la ruta:** antes de acceder al archivo, debe calcularse su ubicación final y comprobar que continúe dentro del directorio autorizado.

- **Rechazar caracteres NUL:** las entradas que contengan caracteres nulos o representaciones equivalentes como %00 deben ser rechazadas antes de utilizarse en operaciones relacionadas con archivos.
- **Validar después de decodificar:** las entradas deben ser completamente decodificadas y normalizadas antes de aplicar las comprobaciones de seguridad.
- **Mantener actualizado el entorno de ejecución:** deben utilizarse versiones modernas y soportadas del lenguaje, framework y bibliotecas, ya que implementaciones antiguas pueden contener comportamientos inseguros relacionados con caracteres nulos.
- **Aplicar el principio de menor privilegio:** el proceso del servidor web debe tener únicamente los permisos indispensables para acceder a los archivos necesarios para su funcionamiento.